

Simulation of an Autonomous Underwater Vehicle with a Distributed ROS 2 Architecture and *DOOM*-Style First-Person Rendering Engine

Sergio — al428707@uji.es

Universitat Jaume I — Engineering Degree

Course: Aerial and Underwater Systems

February 2026

Abstract

This work describes the design, implementation, and validation of **DOOM AUV Sim**, a comprehensive Autonomous Underwater Vehicle (AUV) simulator built on the ROS 2 Humble middleware and the Python programming language. The system integrates a first-person rendering engine based on 2.5D raycasting —inspired by the classic video game *DOOM* (1993)—, an underwater physics engine with hydrodynamic inertia and battery consumption, an acoustic underwater communication channel with latency and packet loss modeled according to Thorp’s absorption formula, and an onboard autonomy module featuring a control architecture inspired by Brooks’ subsumption paradigm. The distributed architecture employs six ROS 2 nodes interconnected through approximately twenty Pub/Sub topics, without resorting to Services or Actions, a design choice justified by the inherently asynchronous and variable-latency nature of the underwater domain. The system was developed iteratively over multiple design-implementation-test cycles, progressively resolving issues related to depth control stability, robotic arm alignment, realistic acoustic communication, and target capture. The results demonstrate that the simulator enables the autonomous collection of five randomly distributed targets, the reactive avoidance of both dynamic and static obstacles, and the export of the explored map in PCD, PNG, and OBJ formats.

1. Introduction

Autonomous Underwater Vehicles (AUVs) represent one of the most challenging robotic platforms in engineering. Unlike terrestrial or aerial robots, AUVs operate in an environment where conventional electromagnetic communication —including GPS— becomes infeasible beyond a few meters of depth. The standard alternative, acoustic underwater communication, introduces latencies on the order of hundreds of milliseconds to seconds and significant packet loss rates, severely limiting real-time teleoperation capability [1].

In this context, the development of simulators that faithfully reproduce these operational constraints constitutes a critical need both for operator training and for the validation of autonomy algorithms. However, most existing AUV simulators —such as UUV Simulator [2] or Stonefish [3]— focus on the physical fidelity of the underwater medium or on integration with planning frameworks like Nav2, but do not necessarily expose the robot’s perceptual and communication limitations to the user in an intuitive manner.

This project proposes an alternative approach: an AUV simulator that uses a first-person rendering engine inspired by the classic *DOOM* (id Software, 1993) to immersively represent the limited perspective of the vehicle’s forward-looking sonar. This 2.5D raycasting, combined with a fog-of-war system, forces the operator to experience the same sensory constraints as the actual AUV, facilitating an intuitive understanding of its capabilities and limitations.

The system was built entirely on ROS 2 Humble, leveraging

its Pub/Sub communication model to implement a distributed architecture of six nodes that clearly separates the responsibilities of simulation, communication, arbitration, autonomy, and visualization. Throughout its iterative development, multiple technical challenges related to control stability, acoustic channel fidelity, and robotic arm coordination were addressed and resolved.

2. Related Work

2.1 AUV Simulation in ROS 2

The ROS 2 ecosystem offers several options for underwater vehicle simulation. The UUV Simulator package [2], originally developed for ROS 1, provides detailed hydrodynamic models in Gazebo, but its configuration complexity and the computational requirements of full three-dimensional physics simulation make it poorly suited for rapid iteration. Stonefish [3] offers a lighter-weight simulator with native ROS 2 support, but still requires URDF models, world files, and complex sensor configuration.

These simulators prioritize physical fidelity over accessibility and rapid prototyping. In contrast, the approach adopted in this work prioritizes **functional fidelity**: the operational effects (latency, packet loss, limited field of view, inertia) are replicated without the need for a full three-dimensional physics simulation.

2.2 Underwater Acoustic Communication

Sound propagation underwater is governed by frequency-dependent absorption, geometric spreading, and variable channel effects. Thorp’s formula [4] provides a practical approximation to the absorption coefficient α in dB/km for a frequency f in kHz:

$$\alpha = \frac{0.11f^2}{1 + f^2} + \frac{44f^2}{4100 + f^2} + 2.75 \times 10^{-4}f^2 + 0.003 \quad (1)$$

The Transmission Loss (TL) combines geometric spreading and absorption:

$$TL = 20 \log_{10}(R) + \alpha \cdot R \cdot 10^{-3} \quad [\text{dB}] \quad (2)$$

where R is the distance in meters. The resulting Signal-to-Noise Ratio (SNR) is computed as $\text{SNR} = SL - TL - NL$, where SL is the source level and NL is the ambient noise level [5].

2.3 Subsumption Architecture

The subsumption architecture, proposed by Brooks [6], organizes robotic control into behavioral layers with strict priority. Lower layers (reactive survival behaviors) can *suppress* the outputs of higher layers (goal-oriented behaviors). This paradigm is particularly well-suited for dynamic and unpredictable environments such as the ocean floor, where obstacle avoidance must take absolute priority over any mission objective.

3. Hypothesis and Design

The central hypothesis of this work is that a **lightweight AUV simulator based exclusively on ROS 2 Pub/Sub communication and 2.5D rendering** can reproduce the essential operational constraints of a real underwater vehicle —acoustic latency, limited perception, hydrodynamic inertia— with sufficient functional fidelity to validate autonomy algorithms and train operators, without requiring the complexity of a full three-dimensional physics simulator.

The design is founded on three premises:

1. **Strict simulation-control separation:** The simulation engine publishes ground truth on isolated topics (`/sim/*`), and a bridge node introduces delay before republishing them as AUV perception (`/auv/*`).
2. **Realistic communication as a requirement, not an option:** All operator commands must traverse the simulated acoustic channel. No direct communication shortcuts exist.
3. **Safety through subsumption:** Obstacle avoidance and battery failsafe behaviors can override both autonomous and manual control.

4. Methodology

4.1 Overall System Architecture

The system consists of six custom ROS 2 nodes, an external visualization node (RViz 2), and a teleoperation node executed in a separate terminal. Figure 1 presents the complete topology.

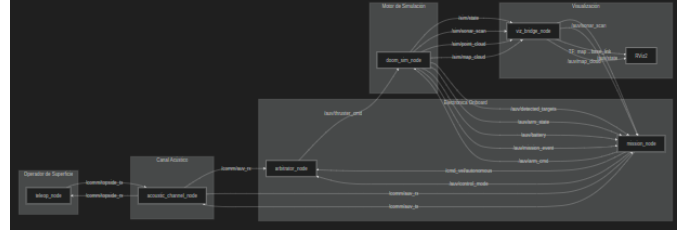


Figure 1: ROS 2 system topology (rqt_graph). Nodes are grouped into five functional domains: Topside, Acoustic Channel, Onboard, Simulator, and Visualization.

Table 1 summarizes the nodes and their responsibilities.

Table 1: ROS 2 system nodes and their roles.

Node	Responsibility
doom_sim_node	Simulation engine: physics, Pygame rendering, sensor publishing
teleop_node	Terminal-based operator interface with dashboard
acoustic_channel_node	Bidirectional channel with latency and packet loss
arbitrator_node	Manual/Auto command multiplexer
mission_node	Onboard autonomy logic with finite state machine
viz_bridge_node	Delayed bridge for RViz 2 (TF + PointCloud)

The decision to use exclusively Pub/Sub topics —without ROS 2 Services or Actions— stems from the fundamentally asynchronous nature of the domain. A ROS 2 Service is blocking: the client waits for the server response. In a scenario where communication may take seconds or be lost entirely, this pattern would cause unacceptable deadlocks in the control loop. Actions, although asynchronous with feedback support, would introduce state management complexity that is more directly resolved through the mission node’s state machine combined with topic-based events.

4.2 Simulation Engine

The `doom_sim_node` runs a main loop at 20 Hz that integrates three subsystems: the underwater physics, the rendering engine, and the ROS 2 interface.

4.2.1 Physics Engine

The physics engine models an AUV with three controllable degrees of freedom: linear advance, yaw rotation, and vertical movement. Rather than directly assigning velocity from received commands, the system applies a model of **cumulative hydrodynamic inertia**. Commands from `/auv/thruster_cmd` (type `Twist`) are interpreted as forces that accelerate the vehicle:

```

1 # Acceleration: commands accumulate velocity
2 self.linear_vel += cmd.linear.x * FORCE_LINEAR
3 self.angular_vel += cmd.angular.z * FORCE_ANGULAR
4 self.z_vel += cmd.linear.z * FORCE_Z
5
6 # Hydrodynamic drag: per-tick decay
7 self.linear_vel *= WATER_DRAG_LINEAR # 0.92
8 self.angular_vel *= WATER_DRAG_ANGULAR # 0.85
9 self.z_vel *= WATER_DRAG_LINEAR

```

Listing 1: Acceleration and hydrodynamic drag model.

The drag factor $d = 0.92$ implies that, in the absence of external forces, velocity decays exponentially as $v(t) = v_0 \cdot d^t$, producing smooth motion with perceptible inertia. Collision is resolved through direct cell-map checking and obstacle proximity verification, including logic that allows the vehicle to escape if it is already within the collision radius.

The battery model implements heterogeneous consumption: a base drain of 0.001 units per tick plus contributions proportional to linear velocity ($|\dot{x}| \times 0.005$), angular velocity ($|\dot{\theta}| \times 0.01$), and vertical velocity ($|\dot{z}| \times 0.01$).

4.2.2 Rendering Engine

The rendering employs the raycasting technique [7], where 120 rays are cast from the AUV’s position across a 60° arc ($\text{FOV} = \pi/3$). For each ray, the algorithm advances iteratively in 5-pixel increments until a wall is detected or the maximum depth (100 units) is reached. The perpendicular distance is computed by applying fisheye correction:

$$d_{\perp} = d_{\text{raw}} \cdot \cos(\theta_{\text{player}} - \theta_{\text{ray}}) \quad (3)$$

The rendered column height is inversely proportional to distance: $h = 21000/(d_{\perp} + \epsilon)$. Dynamic obstacles (vessels, marine creatures) and static obstacles (reefs, mines) are rendered as billboard sprites with type-differentiated shapes. Collection targets are drawn as circles with an orientation indicator (red line).

A side effect of raycasting is the progressive map revelation (fog of war): each cell traversed by a ray is marked as explored in a boolean matrix `fog_map[100×100]`, which controls visibility in both the sonar minimap and the export data.



Figure 2: DOOM AUV simulator interface. The main view shows the 2.5D raycasting rendering. The bottom HUD displays depth, battery, and SNR. The upper-right minimap shows sonar coverage with fog of war. The robotic arm with gripper is visible at the bottom center.

4.3 Acoustic Communication Channel

The `acoustic_channel_node` implements a bidirectional channel between the surface operator (topside) and the AUV’s onboard electronics. Each JSON message entering through `/comm/topside_tx` (downlink) or `/comm/auv_tx` (uplink) is enqueued with a delay computed as:

$$\tau = \frac{R}{c_s} + U(0.01, 0.05) \quad [\text{s}] \quad (4)$$

where R is the distance to the AUV, $c_s = 1500$ m/s is the speed of sound in water, and $U(a, b)$ is a uniform random component modeling processing jitter. Additionally, each packet has a 5% probability of loss, independently and identically distributed.

The `CommunicationPhysics` module implements the complete acoustic SNR model according to Equation 2 with Thorp absorption (Equation 1) at $f = 25$ kHz and $SL = 185$ dB re $1 \mu\text{Pa}$. The optical SNR is modeled with an exponential attenuation ($c = 0.08 \text{ m}^{-1}$, range 100 m) corresponding to clear oceanic water. A simplified scintillation model introduces random 5% signal drops simulating the effect of internal waves.

4.4 Control Architecture

4.4.1 Command Arbitration

The `arbitrator_node` operates as a temporal multiplexer: at each tick of its 20 Hz loop, it publishes to `/auv/thruster_cmd` the most recent command received from the active source (manual or autonomous), provided that

command is recent (< 0.5 s). If no recent commands are available, it publishes a null `Twist`, which the hydrodynamic drag translates into progressive deceleration.

4.4.2 Safety Subsumption

The `mission_node` implements a priority hierarchy inspired by the subsumption architecture [6]. The safety layers operate **in all modes**, including manual, and can force a switch to autonomous mode:

1. **EMERGENCY_REVERSE** ($d < 1.5$ m): Pure reverse for 2 s. Highest priority.
2. **AVOID_CRITICAL** ($d < 3.0$ m): Reverse with rotation toward the side with more free space, for 3 s.
3. **BATTERY_FAILSAFE** (battery $< 10\%$): Forces emergency ascent.
4. **Mission states** (EXPLORE, APPROACH, ALIGN, etc.): Active only in AUTO mode.

The evasion direction is determined in real time by comparing the sum of sonar ranges in the left and right hemifields:

```
1 mid = len(scan.ranges) // 2
2 left_sum = sum(scan.ranges[mid:])
3 right_sum = sum(scan.ranges[:mid])
4 turn_dir = 1.0 if left_sum > right_sum else -1.0
```

Listing 2: Evasion direction determination.

4.5 Autonomous Mission

The onboard autonomy is structured as a finite state machine with six main states (Figure 3).

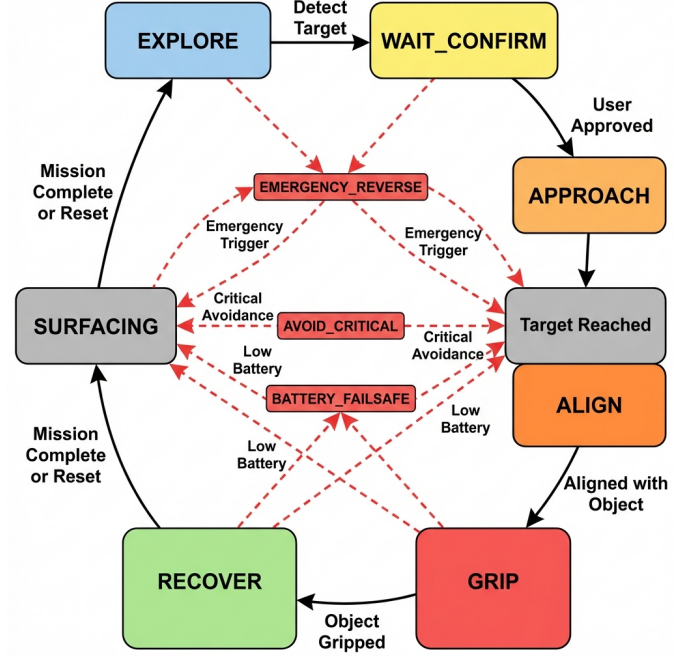


Figure 3: Autonomous mission finite state machine. Safety transitions (red lines) can interrupt any state.

EXPLORE: The AUV follows a lawnmower pattern (systematic zigzag sweep) defined by a list of waypoints on a grid with 20-unit spacing. Navigation employs a proportional yaw controller with gain $K_p = 0.5$, saturated at ± 1.0 rad/s. Linear velocity is reduced to 0.2 when the angular error is large ($|\psi_e| > 0.5$ rad) to prevent advance during sharp turns.

WAIT_CONFIRM: When the simulator publishes a visible target on `/auv/detected_targets`, the `mission_node` transitions to a confirmation wait state, publishing updated telemetry. The operator must send a confirmation (`confirm`) or rejection (`reject`) through the acoustic channel. Rejected targets are stored to prevent re-detection.

4.5.1 Approach Controller (APPROACH)

The APPROACH state implements a **dual proportional controller** that simultaneously governs the AUV's linear and angular velocities, with the objective of converging on the target by minimizing the Euclidean distance $d = \sqrt{\Delta x^2 + \Delta y^2}$ and the heading error ψ_e .

The heading error is computed as the difference normalized to the interval $(-\pi, \pi]$:

$$\psi_e = (\text{atan2}(\Delta y, \Delta x) - \psi_{\text{auv}} + \pi) \bmod 2\pi - \pi \quad (5)$$

where $\psi_{\text{auv}} = 2 \cdot \text{atan2}(q_z, q_w)$ is extracted from the orientation quaternion. The angular control law applies:

$$\omega = \text{clamp}(\psi_e \cdot K_\omega, -\omega_{\text{max}}, \omega_{\text{max}}) \quad (6)$$

with $K_\omega = 0.3$ and $\omega_{\max} = 0.5$ rad/s. This gain was reduced from the original value of 0.5 to prevent aggressive turns that caused overshoot and oscillations around the target.

The linear velocity implements a **switching scheme with angular dead zone**:

$$v = \begin{cases} 0 & \text{if } |\psi_e| > 0.2 \text{ rad} \\ \text{clamp}(0.3 \cdot d, 0.1, 0.5) & \text{otherwise} \end{cases} \quad (7)$$

The switching at $|\psi_e| = 0.2$ rad is critical: **the AUV does not advance unless it is pointing at the target**. This prevents lateral drift which, given non-zero inertia, would accumulate velocity perpendicular to the desired heading. The distance-proportional velocity ($0.3 \cdot d$) provides natural deceleration in the final phase, preventing overshoot.

The transition to ALIGN is triggered when three conditions are jointly satisfied: $d < 1.5$ m, $|\Delta z| < 2.0$ m (depth), and $|\psi_e| < 0.3$ rad. An emergency brake ($v = 0$) is applied unconditionally at $d < 0.5$ m to prevent collision with the target.

4.5.2 Alignment and Visual Servoing (ALIGN)

The ALIGN state constitutes the most complex control module in the system. Unlike APPROACH, which operates in the world’s Cartesian space, ALIGN employs **Image-Based Visual Servoing (IBVS)**: the error is defined directly in the screen coordinates (u, v) of the rendered target, and the control action is applied to the robotic arm actuators.

Body freeze. Upon entering ALIGN, the AUV completely halts all body motion ($v = 0, \omega = 0$). This design decision avoids an undesirable feedback loop: any body rotation displaces the target image on screen, which would trigger arm corrections, which in turn would alter the relative position. Keeping the body static decouples the two control systems.

Arm control. The arm is controlled through discrete commands (“left”, “right”, “up”, “down”, “rot_left”, “rot_right”) published at 10 Hz. To determine which command to issue, the algorithm implements a **priority policy with interleaving**:

```

1 # 1. Rotation - HIGH priority
2 if abs(diff_deg) >= 5.0:
3     if abs(diff_deg) >= 15.0 or tick_even:
4         cmd = "rot_right" if diff_deg > 0
5         else "rot_left"
6 # 2. Screen X position
7 elif abs(cur_x - target_x) > 5: # px
8     cmd = "right" if cur_x < target_x
9     else "left"
10 # 3. Screen Y position
11 elif abs(cur_y - target_y) > 5: # px
12     cmd = "down" if cur_y < target_y
13     else "up"
14 else:
15     pos_aligned = True

```

Listing 3: ALIGN state visual servoing loop.

Gripper rotation is computed as the normalized angular difference between the target’s absolute orientation (extracted from its quaternion) and the gripper’s current angle:

$$\Delta\theta = (\theta_{\text{target}} - \theta_{\text{gripper}} + 180^\circ) \bmod 360^\circ - 180^\circ \quad (8)$$

The interleaving policy (`tick_even`) ensures that when the rotation is “close enough” ($|\Delta\theta| < 15^\circ$) but not “perfect” ($|\Delta\theta| \geq 5^\circ$), the system alternates between rotation and position corrections on even and odd ticks. This prevents the arm from becoming locked in a purely rotational correction cycle.

Creep forward. Only when both arm position and rotation are aligned but the 3D distance to the target exceeds 1.2 m does the system apply a minimal linear advance of $v = 0.1$, bringing the body closer to the object without losing alignment. This logic functions as a proximity bang-bang controller.

Table 2 summarizes the control parameters for both states.

Table 2: APPROACH and ALIGN controller parameters.

State	Parameter	Value	Unit
APPROACH	K_ω (heading)	0.3	–
APPROACH	$\omega_{\max}$	0.5	rad/s
APPROACH	v range	[0.1, 0.5]	m/s
APPROACH	Angular dead zone	0.2	rad
APPROACH	Transition dist.	1.5	m
ALIGN	Rotation tol. (strict)	5.0	°
ALIGN	Rotation tol. (interleave)	15.0	°
ALIGN	Screen position tol.	5	px
ALIGN	Creep v	0.1	m/s
ALIGN	Grip dist. (d_{3D})	1.2	m

4.5.3 Grip Verification and Closure (GRIP)

The ALIGN-to-GRIP transition is triggered when three conditions are simultaneously true: $|\Delta\theta| < 5^\circ$, screen-centered position ($|u_e| < 5$ px and $|v_e| < 5$ px), and $d_{3D} < 1.2$ m. After the transition, the rendering module independently verifies three conditions: 3D distance < 1.5 m, gripper angular error $< 30^\circ$, and on-screen distance < 50 px. If capture fails, the system returns to ALIGN step 0 and reopens the gripper for a retry.

4.6 Procedural Environment

The MapManager generates a procedural underwater world on a 100×100 cell map. Table 3 details the obstacle distribution.

Table 3: Obstacle and target distribution.

Type	N	Depth (m)	Dynamic	Color ID
Vessels	10	0–10	Yes	Red (3)
Creatures	10	40–60	Yes	Blue (4)
Reefs	15	17–37	No	Brown (5)
Mines	15	70–90	No	Gray (6)
Targets	5	5–95	No	Cyan

5. Iterative Development Process

The system was developed through a continuous iterative refinement process spanning over fifteen development cycles. The following sections document the main milestones and technical blockers resolved, organized chronologically.

5.1 Phase 1: Base Architecture

The first iteration established the fundamental components: the simulation node with raycasting rendering, the *teleop_node* with direct communication via *Twist* topics, and a static map with fixed walls. In this phase, manual control published directly to the */cmd_vel* topic without an intermediate acoustic channel. The physics assigned velocity instantaneously, without inertia.

5.2 Phase 2: Acoustic Communication and Dashboard

The *acoustic_channel_node* was introduced as a mandatory intermediary for all operator-AUV communication. The *teleop_node* was redesigned to package commands as JSON and transmit them via */comm/topside_tx*. The terminal dashboard was implemented using ANSI codes to display real-time telemetry.

A significant blocker was the detection of **acoustic link disconnection**: the dashboard displayed “DISCONNECTED” intermittently because the last-packet timestamp verification was not updating correctly. The solution consisted of updating *last_packet_time* upon reception of *any* valid JSON, regardless of its content.

5.3 Phase 3: Autonomy and Obstacle Avoidance

The implementation of the *mission_node* with the state machine and subsumption architecture required multiple iterations. The first critical issue was that the **autopilot did not assume control** when needed: the safety layers failed to force AUTO mode in emergency situations. The solution was to add explicit publishing to */auv/control_mode* from the safety layers.

5.4 Phase 4: Arm Control and Stabilization

The robotic arm alignment phase for target capture proved to be the most problematic and required at least six refinement cycles:

- **Misaligned gripper:** The gripper consistently positioned itself to the left of the target. The visual servoing logic was refined to balance rotation and positional adjustment.
- **Capture distance:** Initially set at 3 m, it proved excessive. It was iteratively reduced to 1.5 m (verification in *renderer.py*) and 1.2 m (transition in *mission_node*).
- **Vertical oscillation:** The depth controller with gain $K_p = 1.0$ caused vertical oscillations that prevented capture. The gain was reduced to $K_p = 0.3$ for smoother control.
- **Arm speed:** Rotation and translation speeds of the arm were adjusted to allow precise control without overshoot.
- **Non-closing gripper:** The angular tolerance was relaxed from 5° to 30° to allow capture with approximate alignments, which is more operationally realistic.

5.5 Phase 5: Acoustic Realism and 3D Export

In the final phase, residual direct communication channels were eliminated, ensuring that 100% of manual control passed through the acoustic channel with latency and packet loss. Map export in Wavefront OBJ format as a 3D mesh was implemented, complementing the existing PCD and PNG formats.

6. Testing and Results

6.1 Test Protocol

Tests were conducted on an Ubuntu 22.04 LTS environment with ROS 2 Humble Hawksbill, Python 3.10, and Pygame 2.x. The system is launched via the launch file:

```
1 $ colcon build
2 $ source install/setup.bash
3 $ ros2 launch doom_auv_sim doom_sim.launch.py
4 # In a separate terminal:
5 $ ros2 run doom_auv_sim teleop_node
```

Listing 4: Execution sequence.

6.2 Functional Results

Table 4 summarizes the results obtained from the complete system tests.

Table 4: Functional test results.

Metric	Value	Status
Targets collected (auto)	5/5	✓
Obstacle avoidance	Functional	✓
Battery failsafe	Active < 10%	✓
Acoustic communication	Delay + 5% loss	✓
Fog of war	Progressive	✓
PCD/PNG/OBJ export	Post-mission	✓
Simulation frequency	20 Hz stable	✓
Control frequency	10 Hz stable	✓

6.3 Depth Control Analysis

Reducing the depth controller gain from $K_p = 1.0$ to $K_p = 0.3$ eliminated the vertical oscillations observed in previous versions. The steady-state depth error remains below ± 0.5 m, which is sufficient for the APPROACH-to-ALIGN transition (tolerance of 2.0 m).

6.4 Acoustic Channel Analysis

The observed mean delay ranges between 70 and 300 ms depending on the distance from the starting point. The 5% packet loss rate introduces noticeable jumps in the telemetry dashboard without compromising system stability, thanks to the asynchronous design of the architecture.

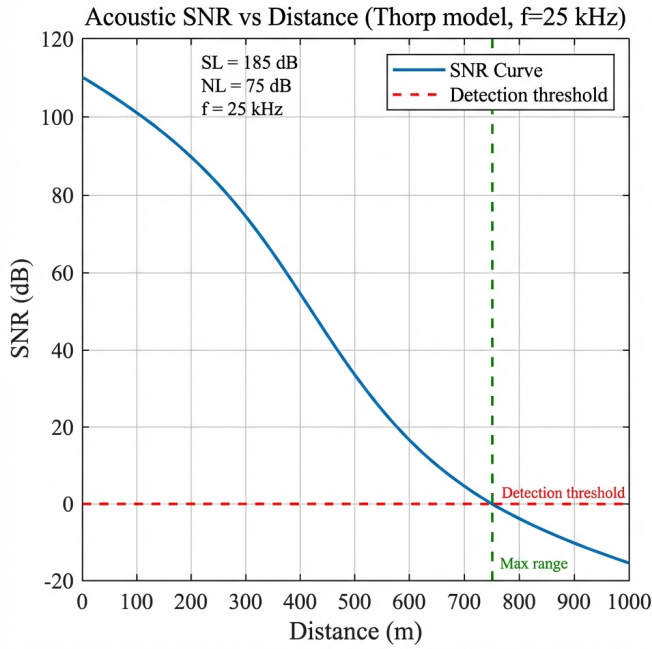


Figure 4: Acoustic SNR computed as a function of distance using Thorp’s absorption formula at 25 kHz.

7. Conclusions and Future Work

This work has demonstrated the viability of a lightweight AUV simulator built on ROS 2 that reproduces the essential operational constraints of the underwater domain. The system successfully integrates a raycasting rendering engine, an acoustic communication channel with a Thorp-based propagation model, a control architecture with subsumption hierarchy for safety, and an onboard autonomy module capable of completing exploration and collection missions.

The iterative development process highlighted the importance of parameter tuning in robotics: the stability of depth control, gripper capture tolerance, and robotic arm speed required multiple empirical adjustment cycles. The documentation of these cycles constitutes a valuable pedagogical contribution, as it illustrates the reality of robotics development beyond theoretical architecture.

7.1 Identified Limitations

- **Reactive navigation:** The absence of a global path planner (e.g., Nav2 with A*) limits the AUV’s ability to circumnavigate complex obstacles. Straight-line navigation with temporal evasion may result in suboptimal trajectories or deadlocks in adverse geometric configurations.
- **i.i.d. packet loss:** The independent loss model does not capture the temporal correlation observed in real acoustic channels. A Markov model (Gilbert-Elliott) would provide greater realism.

- **2D path planning:** Both Nav2 and most ROS 2 planners are designed for 2D navigation, which limits their direct applicability to an AUV operating in three dimensions.

7.2 Future Work

- Integration of Nav2 for horizontal 2D trajectory planning while retaining the current depth controller.
- Correlated acoustic channel model (Gilbert-Elliott) and active application of Doppler shift to control data.
- Extension to multi-AUV cooperative scenarios with TDMA acoustic channel access protocols.
- Addition of ocean current models affecting both navigation and acoustic propagation.
- Integration with `slam_toolbox` for simultaneous mapping from the simulated sonar data.

References

- [1] M. Stojanovic and J. Preisig, “Underwater Acoustic Communication Channels: Propagation Models and Statistical Characterization,” *IEEE Commun. Mag.*, vol. 47, no. 1, pp. 84–89, 2009.
- [2] M. M. M. Manhaes, S. A. Scherer, M. Voss, L. R. Douat, and T. Rauschenbach, “UUV Simulator: A Gazebo-based package for underwater intervention and multi-robot simulation,” *Proc. IEEE OCEANS*, 2016.
- [3] M. Cieřlak, “Stonefish: An Advanced Open-Source Simulation Tool Designed for Marine Robotics, with a ROS Interface,” *Proc. IEEE OCEANS*, 2019.
- [4] W. H. Thorp, “Analytic Description of the Low-Frequency Attenuation Coefficient,” *J. Acoust. Soc. Am.*, vol. 42, no. 1, p. 270, 1967.
- [5] R. J. Urick, *Principles of Underwater Sound*, 3rd ed. Peninsula Publishing, 1983.
- [6] R. A. Brooks, “A Robust Layered Control System for a Mobile Robot,” *IEEE J. Robot. Autom.*, vol. 2, no. 1, pp. 14–23, 1986.
- [7] Lode Vandevenne, “Raycasting Tutorial,” <https://lodev.org/cgtutor/raycasting.html>, accessed Feb. 2026.
- [8] Open Robotics, “ROS 2 Documentation: Humble Hawksbill,” <https://docs.ros.org/en/humble/>, 2022.
- [9] M. Quigley *et al.*, “ROS: an open-source Robot Operating System,” *Proc. ICRA Workshop on Open Source Software*, 2009.
- [10] S. Macenski *et al.*, “The Marathon 2: A Navigation System,” *Proc. IEEE/RSJ IROS*, 2020.